

Low-Resource Sentiment Analysis on the 2025 Steam Dataset Using Hierarchical Navigable Small World and Vocabulary of Domain-specific Sentiment N-grams

Jethro Elijah F. Bolima

Abstract

Understanding consumer sentiment is crucial for a business to make decisions on how to continue their product or service. For games, a rich corpus of reviews can be found in Steam, a game distribution platform. In this paper, word embeddings are used for sentiment analysis and topic modelling. Hierarchical Navigable Small World indexing is used to enable performance in a low-resource environment, in particular a 16 GB RAM and 4 GB VRAM laptop. Class-based term frequency-inverse document frequency is also utilized to obtain and use a vocabulary of domain-specific sentiment n-grams. This paper has managed to reduce the number of output embedding dimensions only to 4, supplemented by 5 other domain-specific features via feature extraction. The best model using the suggested approach achieved 93.49% macro-averaged f1-score on a 4:1 positive-negative review ratio with 544,744 samples.

I. Introduction

The Global Online Gaming Market was valued at US\$ 169.26 billion in 2025 and is set to witness steady growth and is expected to reach US\$ 388.10 billion by 2033 [1]. Digital PC game storefront Steam is one of the most well-known online gaming platforms and distributors worldwide and in 2024, Steam was estimated to have sold over 718 million games in total [2] and has definitely contributed to the gaming market as it is today. The democratization of publishing and selling games Steam allowed in their platform has ushered a boom of independent game developers and removed the necessity of the AAA branding to sell games well. As the market has become saturated and more competitive, understanding the sentiment of gamers remains important.

Laptops are the most common device Filipinos are using for computer-related activities in 2024 [3]. Considering the average annual Filipino family income is Php 353,230 or approximately Php 29,435.83 per month [4], it can be inferred that the usual laptop Filipinos have are budgeted at Php 20,000 - Php 50,000. Laptops on this price range usually have 16 GB RAM and integrated or 4 GB VRAM. The problem occurs should an average Filipino student or researcher try to optimize their models using a dataset with hundreds of thousands of samples and a very wide

dimension size, e.g. a corpus of Steam reviews and its embeddings; the process will take too long or crash their application.

This paper approaches the Steam review dataset on the memory constraint of budget laptops with the three-step process for sentiment analysis:

- Finetuning lightweight embedding models
- Get k-nearest neighbors distances and indices using Hierarchical Navigable Small World and represent the entire text embedding of reviews with neighborhood statistics
- Supplement the model with domain-specific features; this paper used class-based TF-IDF to extract a vocabulary of “positive” and “negative” n-grams and checked the frequency of usage of the identified n-grams in the reviews. Behavioral metrics were also included as it was available in the dataset.

The three-step process allows analysis using lightweight text embeddings to have competitive evaluation metrics on a properly HSNW indexed training set. While there is an overhead of creating the index, querying the embeddings is quick and the recall rate is high [5].

Relatively light and fast. These factors allow budget laptops to scale the number of dimensions down to a workable number of neighbors k.

Improved accuracy. Since the paper is using lightweight and out-of-the-box (OOTB) models, domain-specific features augment lost information not identified by the embeddings.

A similar process is used for topic modelling. BERTopic is a topic model that approaches topic modelling as a clustering task and extends this process by extracting coherent topic representation through the development of a class-based variation of TF-IDF [6]. This paper leverages on the modularity of the BERTopic pipeline which consists of the following:

- Embedding documents
- Reducing dimensionality of embeddings
- Clustering reduced embeddings into topics
- Tokenization of topics
- Weight tokens
- Represent topics with one or multiple representations

II. Related Work

- Abinmayu and Mahendra achieved a macro-averaged F1-score of 0.9317 and accuracy of 0.9320 using Chi-Square feature selection, TF-IDF, and SVM on 5,000 Steam game reviews [7].
- Sattarov et al. mana achieved an F1-score of 90% on 4,022 reviews, although they have not stated if the F1-score is macro-averaged or weighted-averaged, with a “gradient-boosting” classifier [8].
- Guzsvinecz and Szűcs managed to analyze 35,983,481 Steam reviews and classified the reviews per emotion using the lexicon-based emotion dictionary NRC [9].

- Naoman et al. used BERT to achieve an F1-score of 0.71 on negative reviews with 356 support, 0.94 on positive reviews with 1,776 support [10]. The authors also used RoBERTa with improved results of 0.78 F1-score on negative reviews with 673 support and 0.95 F1-score on positive reviews with 3,572 support.
- Oktarian et al. achieved 83% accuracy and 85% weighted F1-score using attention-based Bidirectional LSTM on a 50,000 review sample size
- Fadhlurrahman et al. used BERT, Bidirectional LSTM, and CRF to achieve an F1-score of 95.2%, not stated if macro-averaged or weighted-average, on 7,002 reviews.

As of time of writing, none has yet performed sentiment analysis with all these conditions at the same time:

- a Steam review dataset with 500,000 samples and above
- using embedding models for sentiment analysis
- an environment with a constraint of 16 GB RAM and 4 GB VRAM.

III. Dataset

Donald Fountain created the “2025 Steam Dataset: A Large-Scale, Multi-Modal Dataset of the Steam Gaming Platform” [13]. Outlined below is the data collection methodology employed:

- Exclusive use of official Steam Web API
- Collection Period: August-September 2025
- API Rate Limiting: Conservative 1.5s delays (17.3 requests/minute)
- Success Rate: 56% (remaining failures due to delisted content, regional restrictions)
- Embedding Model: BGE-M3 (1024 dimensions, multilingual support for 100+ languages)
- Quality Assurance: Multi-phase validation with comprehensive documentation
- Infrastructure: Proxmox Astronomy Lab enterprise platform with PostgreSQL 16.10 and NVIDIA A4000 GPU

The scale and scope of the dataset are as follows:

- 239,664 total applications across all content types (games, DLC, software, tools, videos)
- 1,048,148 user reviews with full text and metadata
- 101,226 unique developers and 85,699 unique publishers
- Coverage from 1997 to 2025 (28 years of platform history)

This paper will use 544,744 samples focused on user reviews limited to English or Emoticon-only, comprising 133,443 negative reviews, and 411,301 positive reviews. The cleaning process will be explained in the following chapter.

IV. Methodology

This paper has two parts: Sentiment Analysis and Topic Modelling. Both parts utilize the same process with slight modifications depending on the part.

Feature Selection

The dataset is stored in a PostgreSQL database and the pre-cleaned data is streamed into Jupyter Notebook via the *psycopg* package.

In order to capture as much context necessary for the sentiment of a review, we select from the dataset readily-created vectors of out-of-the-box BGE-M3 embeddings for clustering semantically similar reviews with 1,024 dimensions and 3 behavioral metrics: the number of votes the review has for being funny, the number of votes the review has for being helpful, and the author's playtime at review.

Going forward, we will replace the BGE-M3 embeddings with outputs from finetuned lightweight embedding models.

Data Preprocessing

The dataset consists of 1,048,148 user reviews. While these reviews can be filtered according to their language, some reviews tagged as English are still Japanese, Korean, etc. i.e. of a different language. We used the *fast_langdetect* package and regex for removing reviews with non-english characters. We also removed reviews containing only 1 character as reviews like "a" or "g" only written lack semantic meaning.

After cleaning, there are only 544,744 english and emoticon-only reviews remaining with 133,443 negative reviews, and 411,301 positive reviews as stated earlier.

Part 1: Sentiment Analysis

Fig. 1. shows the process flowchart of finding the best model for sentiment analysis in this paper.

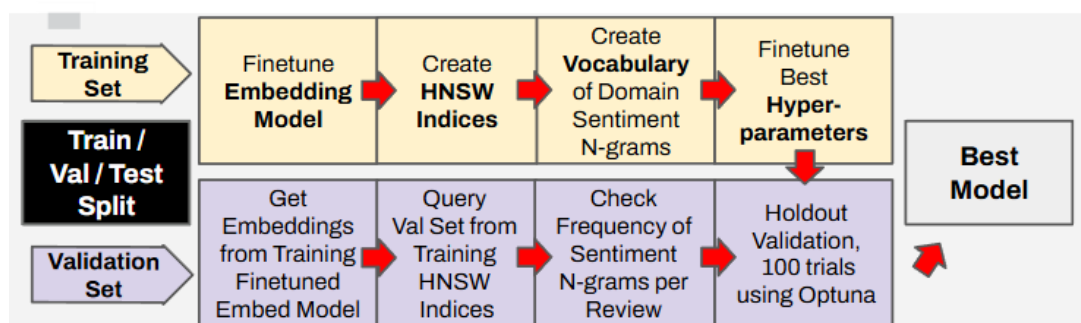


Figure 1. Suggested Sentiment Analysis Process Flowchart

A. Train / Validation / Test Split

We used an 80/10/10 split for the training set, validation set, and test set respectively. We have done the split before properly starting the sentiment analysis process in order to prevent data leakage.

B. Finetuning Lightweight Embedding Models

We will use 2 lightweight embedding models chosen primarily by the number of the model's output dimensions due to memory constraints: *all-MiniLM-L6-v2* and *BGE-small-en-v1.5*, both having 384 output dimensions and BERT as their base model. The difference is that the BGE model has a max context window of 512 tokens, 33.4 million parameters, and it was trained with contrastive learning with in-batch negative samples and resorted to a big batch size (as large as 19,200) and instruction-based fine-tuning on corpora which was refined with a compound data cleaning strategy and selection [14]. *all-MiniLM-L6-v2* only has a max context window of 256 tokens, 22.7 million parameters, and trained with contrastive learning via cosine similarity with all pairs in a 1024 batch size on datasets, mostly forums, given a weighted probability [15].

Fig. 2. shows the number of words per review within the scope of the study. Since most reviews have less than 128 words and max context windows of the lightweight embedding models are limited, we partitioned half the max tokens on the first half of the review and the other half of the tokens into the last part of the review, e.g. the MiniLM model only takes 256 tokens so the first 128 tokens and the last 128 tokens of the review are taken.

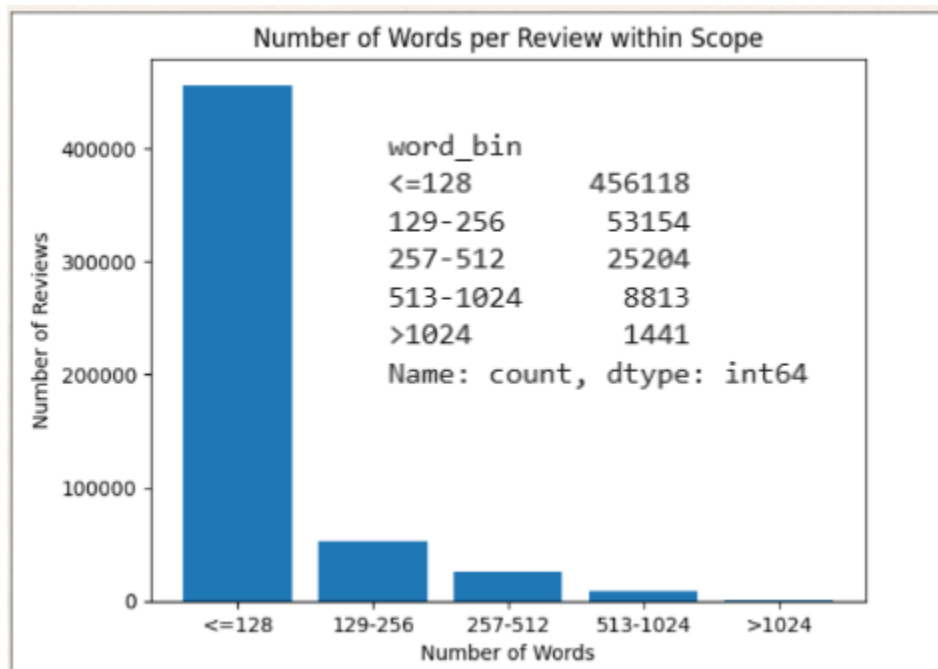


Figure 2. Number of Words per English or Emoticon-only Review

It should be noted that we used Google Colab for finetuning embedding models and encoding the embeddings since we are working with a time constraint. This is still doable on a budget laptop but the batch size should be lower, which results in a much higher training time and also

affects the finetuned embedding model as a whole with respect to the selected loss function. We finetuned both embedding models using the following hyperparameters using SentenceTransformerTrainer:

- 3 epochs
- 64 batch size
- 2 gradient accumulation steps
- learning rate of 2e-5 with cosine scheduler
- warmup steps on 10% of total training set samples
- 0.01 weight decay for regularization
- set with fp16

Since the reviews already come with a label if the reviewer recommends or does not recommend a game, we used the loss function BatchAllTripletLoss, which takes a batch with (input, label) pairs and computes the loss for all possible, valid triplets, i.e., *anchor and positive must have the same label, anchor and negative a different label*. [16], with the cosine distance metric and margin of 0.5.

For context specifically for triplet loss:

- Anchor - A sample from the dataset.
- Positive - A sample that is similar to the anchor.
- Negative - A sample that is dissimilar to the anchor.

The objective of triplet loss is to ensure that the anchor-positive distance is smaller than the anchor-negative distance by a margin. Formally, the loss function is defined by the following formula [17].

$$L_{triplet}(z_a, z_p, z_n) = \max(0, ||z_a - z_p|| - ||z_a - z_n|| + m)$$

where:

- z_a, z_p, z_n and, are the embeddings of the anchor, positive, and negative samples, respectively.
- m is the margin that defines the minimum relative separation between the positive and negative pairs.

We skip this part of the process if we only use OOTB embedding models, in our case the BGE-M3.

C. Creating HNSW indices with Training Set, and Querying

Hierarchical Navigable Small World is an approximate K-nearest neighbor search approach which is fully graph-based and incrementally builds a multi-layer structure consisting from a hierarchical set of proximity graphs (layers) for nested subsets of the stored elements [18]. Starting search from the upper layer together with utilizing the scale separation boosts the

performance compared to NSW and allows a logarithmic complexity scaling ($O(\log N)$), which is necessary for querying with a dataset this paper is using.

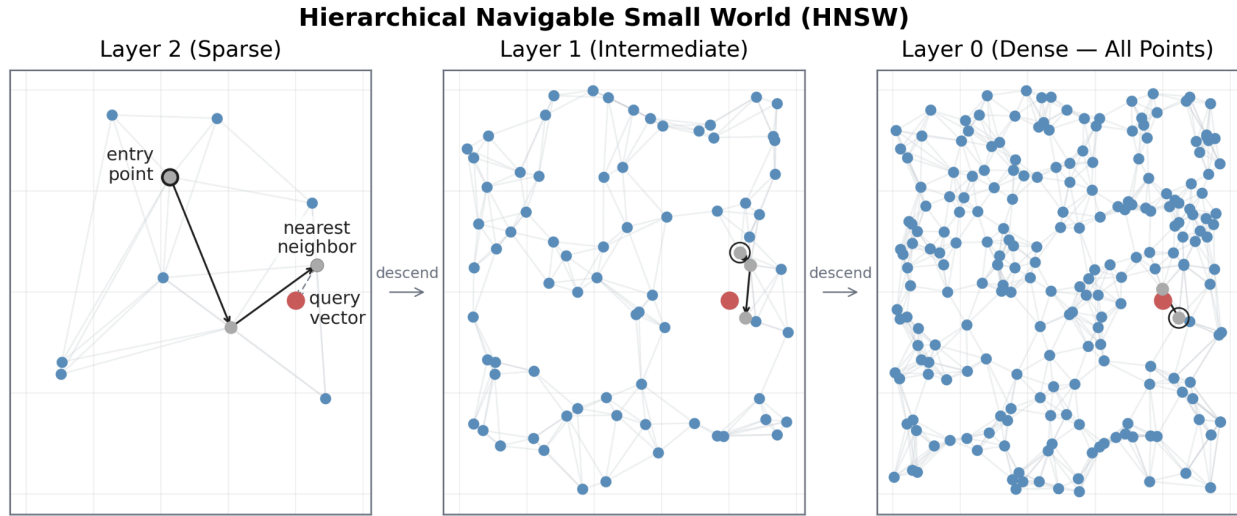


Figure 3. Hierarchical Navigable Small World Multi-Layered Search Process

By Rose electric - Own work, CC BY 4.0, <https://commons.wikimedia.org/w/index.php?curid=174651168>

The HNSW index is a graph-based indexing algorithm that can improve performance when searching for high-dimensional floating vectors. It offers excellent search accuracy and low latency, while it requires high memory overhead to maintain its hierarchical graph structure [5]. We exclusively build the index in the training set. We first normalize the embeddings then, using the *hnswlib* package, set the following parameters for constructing HNSW indices which retains high recall and a relatively fast query time while balancing for the constraints of the environment:

- 32 edges each node have in the graph at each level of the hierarchy
- 128 candidates considered during index construction
- 128 neighbors evaluated during a search

The top 30 neighbors returned after a query. The result of querying new vectors against an HNSW index yields KNN distances and indices for the queried vectors. We will use these distances and indices of the 30 nearest neighbors to represent the text embedding itself, specifically:

- The mean distances to see how tight the neighborhood is
- The standard deviation to see how consistent the review is with its neighbors
- The average sentiment of the training-set-derived HNSW index
- The sentiment consistency of the training-set-derived HNSW index

These 4 engineered features now represent the entire text embedding dimension.

D. Creating and Using the Vocabulary of Domain-specific Sentiment N-grams

Out-of-the-box embedding models or finetuned embedding models with small context windows might lack the necessary domain knowledge in order to process a document more accurately. To address this, we utilize class-based TF-IDF from BERTopic [6] which is formally defined as:

$$W_{x,c} = ||tf_{x,c}|| * \log(1 + \frac{A}{f_x})$$

where:

$tf_{x,c}$ = frequency of word x in class c

f_x = frequency of word x in all classes

A = average number of words per class

The result would be importance scores for words within a cluster. The more important words are within a cluster, the more it is representative of that topic. For sentiment analysis, we will get which n-grams are important for a positive review or a negative review. We removed the stop words after retrieving the top 5% most important unigrams and bigrams per sentiment since doing it in CountVectorizer will remove the stop words before the n-grams are created, e.g. the bigram “not worth” can only be retrieved with the aforementioned process since “not” is a stop word.

E. Finetuning best hyperparameters

HNSW connects all points in the training set so a k-fold cross-validation will cause a data leakage. Instead, since the training and validation set are split from the start, we will do holdout validation with 100 trials in Optuna, an open source hyperparameter optimization framework utilizing the Tree-structured Parzen Estimator algorithm [19] for tuning the hyperparameters. We used the following machine learning algorithms for finding the best model:

- XGBoost
- LightGBM
- HistGradientBoostingClassifier
- MLPClassifier
- Logistic Regression
- GaussianNB

Part 2: Topic Modelling

Fig. 4. shows the process flowchart of modelling topics per sentiment in this paper.

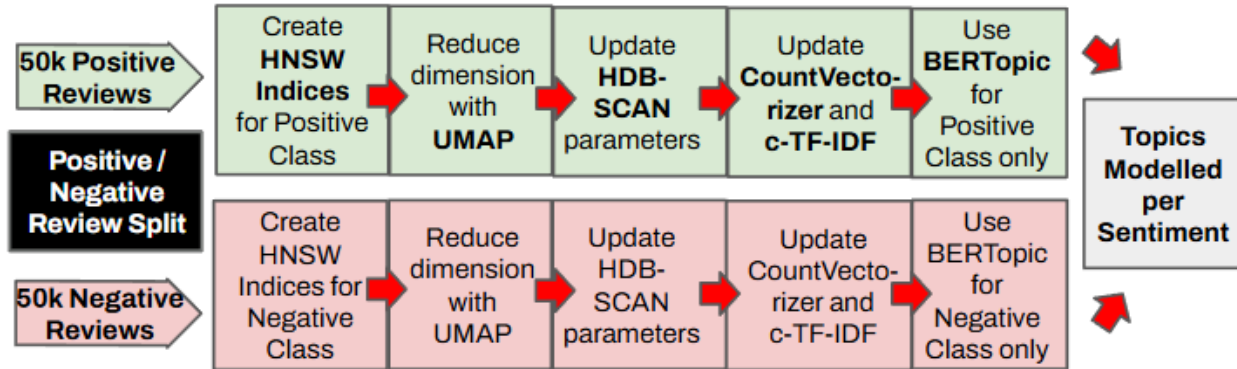


Figure 4. Suggested Topic Modelling Process Flowchart

Since the topics are to be visualized, we sampled 50,000 positive reviews and 50,000 negative reviews which were split from the start.

A. Uniform Manifold Approximation and Projection

HNSW indices were created to get the distances and indices of the top 30 nearest neighbors of each review, which are used for reducing dimensions using Uniform Manifold Approximation and Projection (UMAP).

We chose UMAP since it is a fairly flexible non-linear dimension reduction algorithm [20]. It seeks to learn the manifold structure of the data and find a low dimensional embedding that preserves the essential topological structure of that manifold. We used the following parameters in order to balance the clustering on the high-dimensional embeddings with information in the resulting embeddings:

- 5 components
- 30 neighbors
- Cosine distance metric
- Precomputed KNN from the HNSW index queries

B. Hierarchical Density-Based Spatial Clustering of Applications with Noise

Hierarchical Density-Based Spatial Clustering of Applications with Noise (HDBSCAN) is used for identifying clusters within the UMAP components.

HDBSCAN is a clustering algorithm that is designed to uncover clusters in datasets based on the density distribution of data points [21]. It uses high-density regions to identify clusters and views isolated or low-density points as noise. In order to balance the number of topics to outliers, we used the following parameters:

- Minimum cluster size of 10
- Euclidean distance metric
- Excess of mass cluster selection method

C. BERTopic

Finally, we will leverage on the modularity of BERTopic by using it with the customized parameters of HDBSCAN, CountVectorizer, and c-TF-IDF with the UMAP components to identify the topics of the clusters with the extracted most important words per cluster to get the descriptions of topics.

V. Results

Part 1: Sentiment Analysis

A. Best Embedding Model and Sentiment Classifier

The following table highlights the results of the best models per embedding model.

Embedding Model	OOTB or Finetuned?	Max Tokens and Output Dimensions	Baseline F1-macro with OOTB LightGBM on Text Embeddings only	Best Model F1-macro using Suggested Process
BGE-M3	OOTB	8192 tokens; 1024 dim	89.32%	90.27% - XGBoost
all-MiniLM-L6-v2	Finetuned	256 tokens; 384 dim	91.77%	92.11% - LightGBM
BGE-small-EN-v1.5	Finetuned	512 tokens; 384 dim	93.37%	93.49% - Logistic Regression

Table 1. Best Models per Embedding Model, with baseline and suggested process comparisons

The results have consistently shown that engineering features using the suggested process has increased the predictive power of the models despite extensively reducing the number of dimensions from 1024 or 384 to only 9. The reduction of dimensions also allowed for a faster hyperparameter tuning in a memory-constrained environment. The finetuned smaller embedding models also outperformed the large out-of-the-box BGE-M3 despite the limitations of being lightweight, illustrating the competitive viability of these lightweight models, noting that our resource limitations restrict finetuning solely to this group.

As the best model among all the tested models is Logistic Regression, achieving a macro-averaged F1-score of 93.49%, using the embedding model BGE-small-EN-v1.5, the following are its best hyperparameters and resulting confusion matrix (Fig. 5). Note that the

threshold is updated to 45% after the best parameters are taken during evaluation with the validation set to lower the number of false negatives.

- C: 0.363287283280062
- solver: newton-cholesky
- max_iter: 1000,
- class_weight_positive: 0.5996623900524208
- class_weight_negative: 1

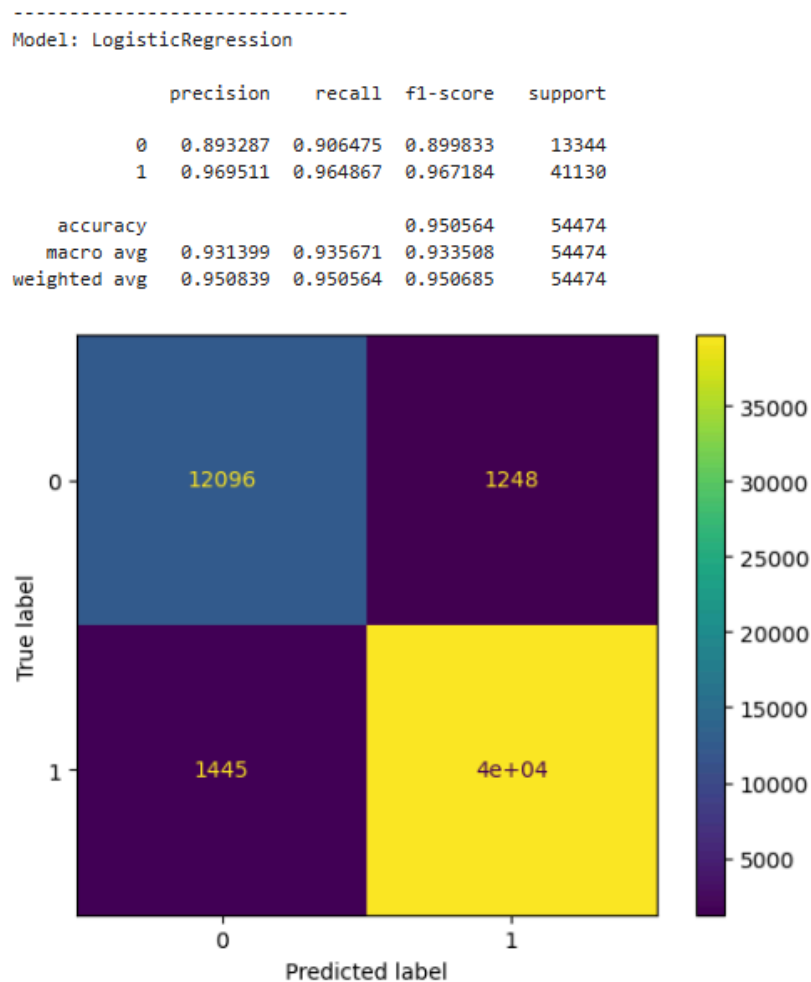


Figure 5. Classification Report and Confusion Matrix of Logistic Regression with BGE-small-EN-v1.5 at 45% Threshold, on the Test Set

B. Embedding Similarity Evaluation

Recall the meaning of anchor, positive pairs, and negative pairs in the context of similarity. To evaluate the similarity of the embeddings, we partitioned the validation set such that 50% consists of positive pairs (evenly split between positive-sentiment and negative-sentiment pairs), and the other 50% consists of negative pairs. We used `BinaryClassificationEvaluator` of `SentenceTransformers` for the evaluation [22]. In particular, we determined the semantic similarity between the text pairs using cosine similarity and used Matthew's correlation coefficient (MCC) afterward to evaluate these scores against the ground truth labels. We also included the accuracy and average precision of the cosine similarity between pairs. Average precision evaluates the model's ranking of embedding similarity scores, while MCC and accuracy evaluate the model's classification strength of similar pairs. The table below shows the performance of each model for this evaluation:

Embedding Model	OOTB or Finetuned?	Average Precision (Cosine Similarity)	Matthew's Correlation Coefficient (Cosine Similarity)	Accuracy (Cosine Similarity)	Best Model F1-macro using Suggested Process
BGE-M3	OOTB	0.6785	0.1215	0.6330	90.27% - XGBoost
all-MiniLM-L6-v2	Finetuned	0.9255	0.7073	0.8537	92.11% - LightGBM
BGE-small-EN-v1.5	Finetuned	0.9469	0.7620	0.8814	93.49% - Logistic Regression

Table 2. Embedding Similarity Evaluation of the selected embedding models on the text-pairing modified validation set

Throughout all the embedding similarity evaluation tests, the finetuned lightweight embedding models consistently scored better against the OOTB BGE-M3. While BGE-small performed the best, the MiniLM-L6 embedding model was the fastest out of the three but still scored competitively, showing the trade-off between speed and performance which might be necessary for memory-constrained environments. This also demonstrates the direct relationship of the model's ranking performance of embedding similarity vs the sentiment analysis predictive performance of the downstream model, i.e. the higher the embedding model's performance is, the better the HNSW index query top-k results will be as the model understands the semantic nuances of the text, which finally leads to a better classification score.

C. On Feature Importance with SHapley Additive exPlanations

In order to understand which features are important for the model's sentiment analysis, we will use SHapley Additive exPlanations (SHAP). SHAP is a game theoretic approach to explain the

output of any machine learning model. It connects optimal credit allocation with local explanations using the classic Shapley values from game theory and their related extensions [23]. For consistency, we will explore the SHAP values beeswarm plot of the XGBoost models using each embedding model.

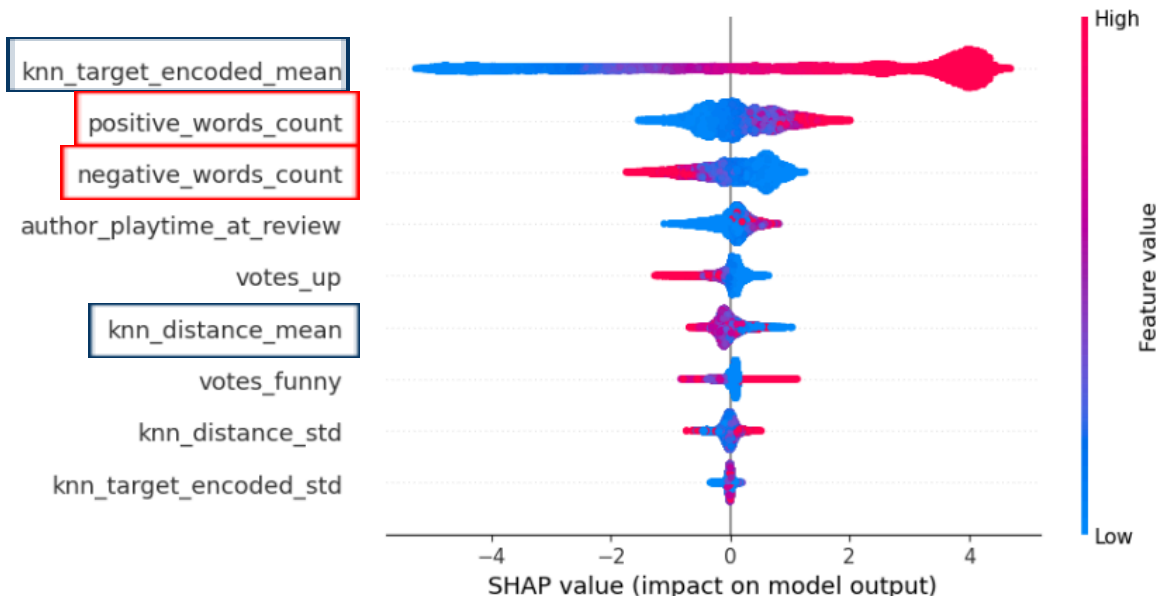


Figure 6. Beeswarm plot of SHAP value for XGBoost on the OOTB BGE-M3 Embeddings

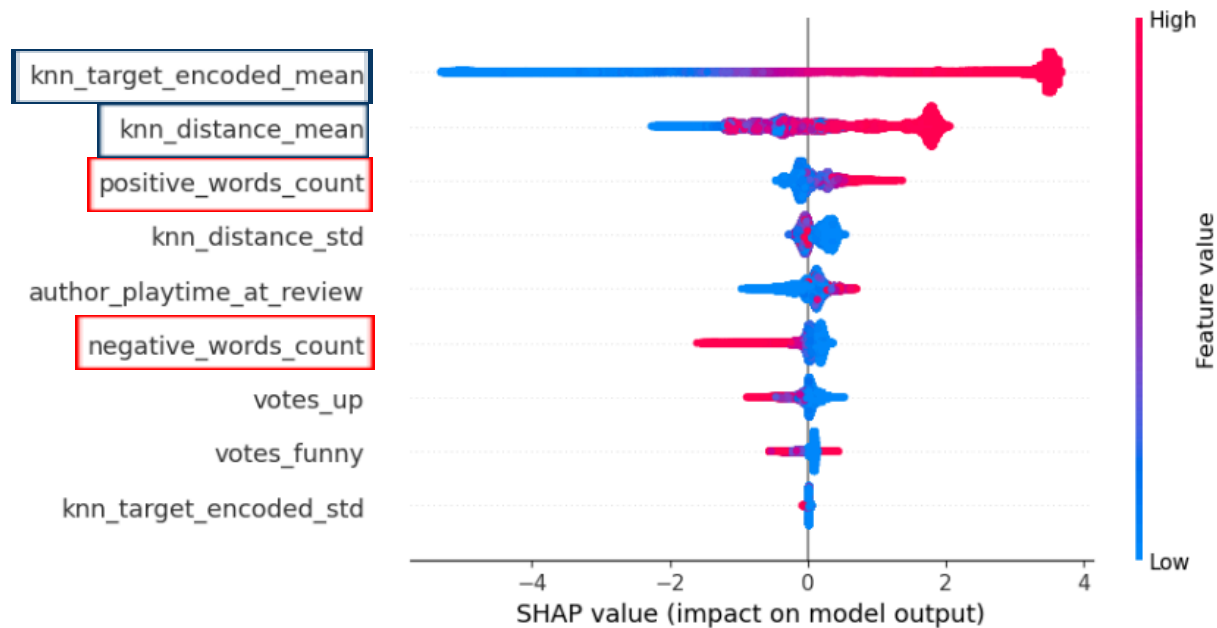


Figure 7. Beeswarm plot of SHAP value for XGBoost on the finetuned all-MiniLM-L6-v2 Embeddings

The beeswarm plots show the successful implementation of the engineered features and its contributions in the model's prediction. `knn_target_encoded_mean` is the average sentiment of the training-set-derived HNSW index, and this highlights how important a good text embedding model is as well as a properly constructed HNSW index, as both are the foundations of the neighborhood statistics features. The beeswarm plots also highlight the importance of domain-specific features for OOTB embedding models or finetuned lightweight models with limited context windows and parameters, as these engineered features successfully supplemented the classification models which lacked domain knowledge. The behavioral metrics also show interesting results, where the model likely flags a review as negative:

- The lower the player's playtime is at review,
- The higher the number of votes the review has

If a review is flagged as funny, the review can be either positive or negative. It can be inferred that players who do not like a game will more likely drop a game as soon as possible then write their review. It can also be inferred that the negative reviews are found to be more helpful as it feels more honest and helps a player know if he will waste time playing the game or not.

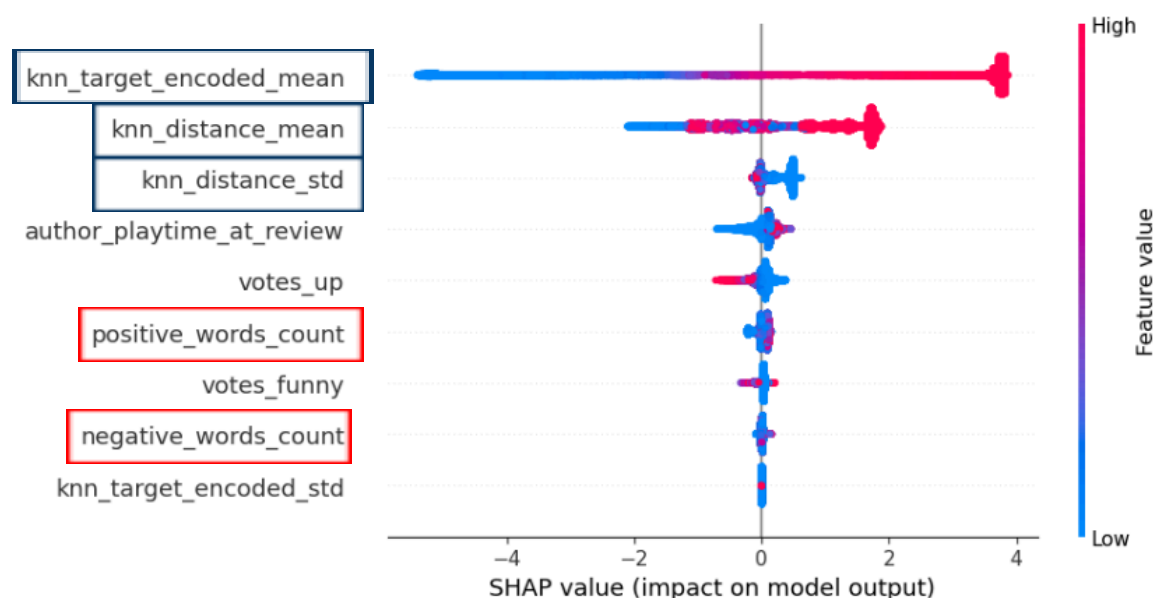


Figure 8. Beeswarm plot of SHAP value for XGBoost on the finetuned BGE-small-EN-v1.5 Embeddings

Recall that the BGE-small-EN-v1.5 has consistently scored the highest in the Embedding Similarity Evaluation. As the embedding model understands the semantic nuances of the reviews more, the less it needs domain-specific features as the embedding model now has better comprehension of the domain. This also shows the direct relationship between the embedding model's ranking of semantic similarity on downstream classification via the HNSW index. As the embedding model performed better, the more important the neighborhood statistics features have become. The behavioral metrics remained the same as they are not part of the embedding nor the vocabulary of the domain.

Part 2: Topic Modelling

Fig. 9 shows the modelled topics derived from the 50,000 sampled positive reviews.

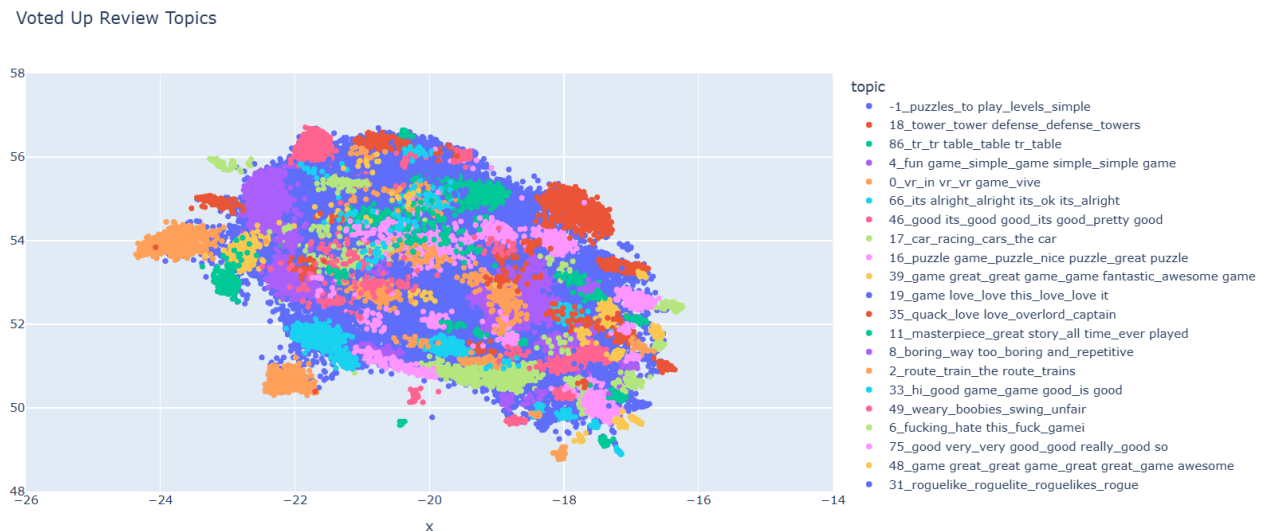


Figure 9. Modelled Topics from the 50,000 Sampled Positive Reviews

There are a total 145 topics for the positive reviews, which can also be interpreted as drivers of positive sentiment within the reviews. Listed below are some of the speculations that were observed from the clusters upon which the developers can also consider for their games:

- Game Aspects
 - Good Soundtrack
 - Availability in Virtual Reality
- Genres and Mechanics
 - Simple Game + Co-op with Friends
 - Mafia-like gameplay + Masterpiece story
 - Roguelike
 - Nostalgic games
 - Visual Novels + Horror
- Niches
 - Train Simulator games

On the other side, Fig. 10 shows the modelled topics derived from the 50,000 sampled negative reviews.

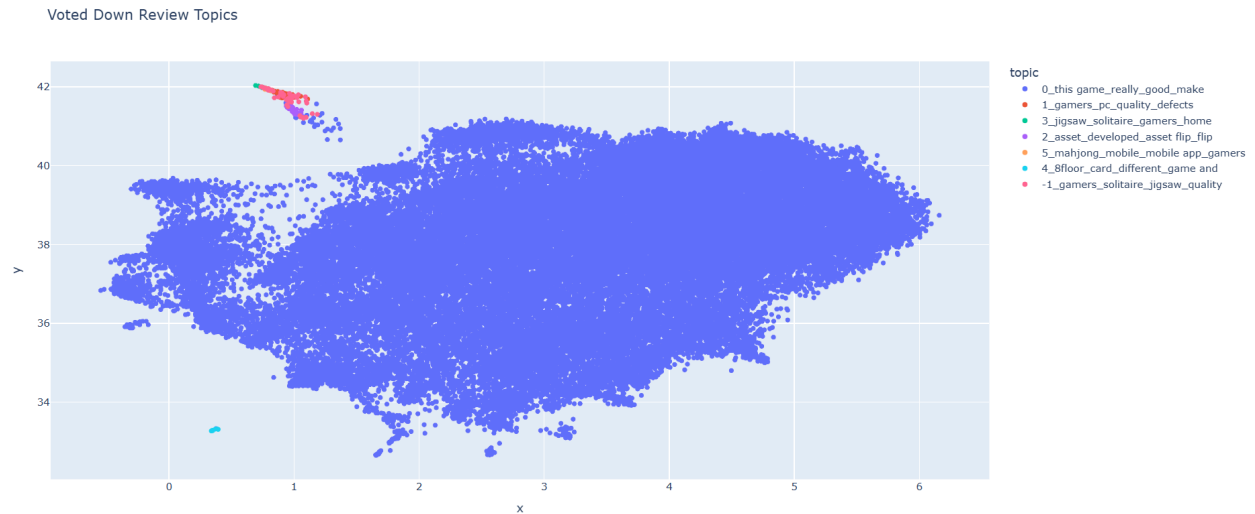


Figure 10. Modelled Topics from the 50,000 Sampled Negative Reviews

There are only a total of 6 topics for the negative reviews, which can also be interpreted as drivers of negative sentiment within the reviews. This shows how dense, how semantically similar the negative reviews are. Listed below are some of the speculations that were observed from the clusters upon which the developers should avoid for their games:

- Egregious usage of AI-generated assets
- Extensive asset flipping
- Low quality / performance

The huge cluster are reviews which contain texts which are semantically similar to “not good”, “not worth”, “not recommend”. It can be inferred that players may write about a lot of things that they like, but when writing a negative review, a few choice words are enough to understand the sentiment.

VI. Conclusion

The paper has demonstrated a process which reduces the load on a memory-constrained environment while still providing competitive and improved results vs the baseline. We have shown that HNSW is effective for representing embeddings through neighborhood statistics. Furthermore, while domain-specific features prove critical for embedding models that lack inherent domain knowledge, their impact yields diminishing returns as the embedding model's semantic ranking performance improves.

This study has shown a direct relationship between embedding similarity evaluation and downstream classification model performance, while also illustrating how behavioral metrics influence sentiment analysis. For the 2025 Steam Dataset, the finetuned BGE-small-EN-v1.5

gave the best embeddings and when paired with Logistic Regression at a 45% threshold, this pipeline achieved the highest classification performance at a 93.49% macro-averaged F1-score.

Finally, the study reveals a highly diverse set of positive sentiment drivers, offering actionable considerations for game developers. Conversely, negative reviews exhibited tight semantic similarity, highlighting critical pitfalls and specific negative sentiment drivers that developers should actively avoid.

VII. Recommendation

The author of the paper considers using product quantization as a next step or an alternate method of approximate nearest neighbors on handling larger datasets. Distance from centroids as a feature should also be considered when using the distances and indices of the nearest neighbors. There are also multiple features and tables found in the 2025 Steam Dataset worth analyzing, either for continuing sentiment analysis or otherwise.

References

- [1] ResearchAndMarkets. (n.d.). *Online Gaming Market Competitive Landscape Report 2025-2033: Top Companies Analysis, Profiles, Strategic Developments, Sustainability Goals, Mergers, Product Launches, Innovations, Revenue Insights [Review of Online Gaming Market Competitive Landscape Report 2025-2033: Top Companies Analysis, Profiles, Strategic Developments, Sustainability Goals, Mergers, Product Launches, Innovations, Revenue Insights]*. Yahoo Finance.
<https://finance.yahoo.com/news/online-gaming-market-competitive-landscape-115700778.html>
- [2] Topic: Steam. (2025). Statista. <https://www.statista.com/topics/4282/steam/>
- [3] Mapa, C. D. (2025, December 22). *Laptop Computer was Most Commonly Used by Filipinos Aged 10 Years and Over in 2024; Sending Messages with File Attachments Topped the List of Computer-Related Activities*. <https://psa.gov.ph/>.
- [4] Philippine Statistics Authority. (2024). *Average Annual Family Income in 2023 is Estimated at PhP 353.23 Thousand* | Philippine Statistics Authority | Republic of the Philippines. [Psa.gov.ph](https://psa.gov.ph).
<https://psa.gov.ph/content/average-annual-family-income-2023-estimated-php-35323-thousand>
- [5] HNSW | Milvus Documentation. (2026). Milvus.io. <https://milvus.io/docs/hnsw.md>

- [6] Grootendorst, M. (2022). *BERTopic: Neural topic modeling with a class-based TF-IDF procedure*. arXiv preprint arXiv:2203.05794.
- [7] Abimanyu, G. R., & Purbolaksono, M. D. (2025, July). *Sentiment Analysis on Game Review on the Steam Platform using Support Vector Machine, TF-IDF and Chi-Square Methods*. In 2025 International Conference on Data Science and Its Applications (ICoDSA) (pp. 1190-1195). IEEE.
- [8] Sattarov, K., Shakhriyorbek, B., Chaudhary, N., Ather, D., Kler, R., & Shukla, B. (2024, July). *Automated sentiment analysis and emotion recognition of Steam game reviews using ensemble machine learning techniques*. In 2024 1st International Conference on Sustainable Computing and Integrated Communication in Changing Landscape of AI (ICSCAI) (pp. 1-10). IEEE.
- [9] Guzsvinecz, T., & Szűcs, J. (2023). *Length and sentiment analysis of reviews about top-level video game genres on the steam platform*. Computers in Human Behavior, 149, 107955.
- [10] Reddy, R., Naoman, A. A., Charan, G. V. S., & Fazal, S. N. (2024, February). *Sentiment analysis of steam reviews using transformer models*. In International Conference on Communications and Cyber Physical Engineering 2018 (pp. 719-727). Singapore: Springer Nature Singapore.
- [11] Oktarian, A. A., Wijaya, F. F., Luthfi, D. R., Muthoharoh, L., Satria, A., & Manullang, M. C. T. (2026). *Enhancing Game Review Sentiment Classification on Steam Platform with Attention-Based BiLSTM*. arXiv preprint arXiv:2605.01315.
- [12] Fadhlurrahman, J. A. M., Herawati, N. A., Aulya, H. R. W., Puspasari, I., & Utama, N. P. (2023, October). *Sentiment analysis of game reviews on steam using bert, bilstm, and crf*. In 2023 International Conference on Electrical Engineering and Informatics (ICEEI) (pp. 1-6). IEEE.
- [13] Fountain, D. (2025). *2025 Steam Dataset: A Large-Scale, Multi-Modal Dataset of the Steam Gaming Platform*. Zenodo. DOI 10.5281/zenodo.17266922
- [14] Xiao, S., Liu, Z., Zhang, P., Muennighoff, N., Lian, D., & Nie, J. Y. (2024, July). *C-pack: Packed resources for general chinese embeddings*. In Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval (pp. 641-649).
- [15] Hugging Face. (n.d.). *sentence-transformers/all-MiniLM-L6-v2* · Hugging Face. Huggingface.co. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>

[16] *Losses — Sentence Transformers documentation.* (2024). Sbert.net.
https://www.sbert.net/docs/package_reference/sentence_transformer/losses.html

[17] *Triplet Loss — Deep Learning.* (2015). Esiee.fr.
<https://perso.esiee.fr/~chierchg/deep-learning/tutorials/metric/metric-2.html>

[18] Malkov, Y. A., & Yashunin, D. A. (2018). *Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs*. IEEE transactions on pattern analysis and machine intelligence, 42(4), 824-836.

[19] *Efficient Optimization Algorithms — Optuna 3.5.0 documentation.* (n.d.).
Optuna.readthedocs.io.
https://optuna.readthedocs.io/en/stable/tutorial/10_key_features/003_efficient_optimization_algorithms.html

[20] *Basic UMAP Parameters — umap 0.5 documentation.* (n.d.).
Umap-Learn.readthedocs.io.
<https://umap-learn.readthedocs.io/en/latest/parameters.html>

[21] GeeksforGeeks. (2024, March 15). *Hierarchical DensityBased Spatial Clustering of Applications with Noise (HDBSCAN)*. GeeksforGeeks.
<https://www.geeksforgeeks.org/machine-learning/hdbscan/>

[22] *Evaluation — Sentence Transformers documentation.* (2020). Sbert.net.
https://www.sbert.net/docs/package_reference/sentence_transformer/evaluation.html

[23] SHAP. (2025). *Welcome to the SHAP Documentation — SHAP latest documentation*.
Shap.readthedocs.io. <https://shap.readthedocs.io/en/latest/>